

10.6.2

RAVULA SHASHANK REDDY - EE25BTECH11047

October 12, 2025

Question

Construct a pair of tangents to a circle of radius 4 cm from a point P at a distance of 7 cm from its centre O .

Solution

The tangent directions $\mathbf{m}$ from an external point $\mathbf{h}$ to the circle

$$g(\mathbf{x}) = \mathbf{x}^\top \mathbf{x} - r^2 = 0 \quad (1)$$

satisfy

$$\mathbf{m}^\top \Phi \mathbf{m} = 0 \quad (2)$$

where

$$\Phi = \mathbf{h}\mathbf{h}^\top - g(\mathbf{h})\mathbf{I} \quad (3)$$

With the point $\mathbf{h} = d\mathbf{e}_1$, we have $g(\mathbf{h}) = d^2 - r^2$. From (3),

$$\Phi = \begin{pmatrix} d^2 & 0 \\ 0 & 0 \end{pmatrix} - (d^2 - r^2) \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad (4)$$

$$= \begin{pmatrix} r^2 & 0 \\ 0 & -(d^2 - r^2) \end{pmatrix} \quad (5)$$

Since Φ is a diagonal matrix, its eigenvalues are:

$$\lambda_1 = r^2, \quad \lambda_2 = -(d^2 - r^2) \quad (6)$$

Solution

The matrix of orthonormal eigenvectors is

$$\mathbf{P} = \mathbf{I}$$

Hence, the unit direction vectors of the tangents are given by:

$$\mathbf{m} = \frac{1}{\sqrt{\lambda_1 - \lambda_2}} \mathbf{P} \begin{pmatrix} \sqrt{-\lambda_2} \\ \pm \sqrt{\lambda_1} \end{pmatrix} \quad (7)$$

From (6),

$$\lambda_1 - \lambda_2 = r^2 - (-(d^2 - r^2)) = d^2 \quad (8)$$

$$-\lambda_2 = d^2 - r^2 \quad (9)$$

Substituting into (7):

$$\mathbf{m} = \frac{1}{\sqrt{d^2}} \begin{pmatrix} \sqrt{d^2 - r^2} \\ \pm r \end{pmatrix} \quad (10)$$

$$= \frac{1}{d} \begin{pmatrix} \sqrt{d^2 - r^2} \\ \pm r \end{pmatrix} \quad (11)$$

Solution

The points of contact are given by

$$\mathbf{q} = \mathbf{h} + \kappa \mathbf{m} \quad (12)$$

Satisfies the circle equation

$$g(\mathbf{h} + \kappa \mathbf{m}) = (\mathbf{h} + \kappa \mathbf{m})^\top (\mathbf{h} + \kappa \mathbf{m}) - r^2 = 0 \quad (13)$$

$$\implies \kappa^2 (\mathbf{m}^\top \mathbf{m}) + 2\kappa (\mathbf{h}^\top \mathbf{m}) + g(\mathbf{h}) = 0 \quad (14)$$

For a tangent, κ has a single repeated root. Since $\mathbf{m}$ is a unit vector ($\mathbf{m}^\top \mathbf{m} = 1$):

$$\kappa = -(\mathbf{h}^\top \mathbf{m}) \quad (15)$$

$$\mathbf{h}^\top \mathbf{m} = (d\mathbf{e}_1)^\top \frac{1}{d} \begin{pmatrix} \sqrt{d^2 - r^2} \\ \pm r \end{pmatrix} = \sqrt{d^2 - r^2} \quad (16)$$

Hence,

$$\kappa = -\sqrt{d^2 - r^2}$$

Solution

The points of contact are:

$$\mathbf{q} = d\mathbf{e}_1 - \sqrt{d^2 - r^2} \frac{1}{d} \begin{pmatrix} \sqrt{d^2 - r^2} \\ \pm r \end{pmatrix} \quad (17)$$

$$= \begin{pmatrix} \frac{r^2}{d} \\ \mp \frac{r \sqrt{d^2 - r^2}}{d} \end{pmatrix} \quad (18)$$

Substituting $r = 4$, $d = 7$:

$$\sqrt{d^2 - r^2} = \sqrt{49 - 16} = \sqrt{33} \quad (19)$$

$$\mathbf{q} = \begin{pmatrix} \frac{16}{7} \\ \mp \frac{4\sqrt{33}}{7} \end{pmatrix} \quad (20)$$

Solution

Thus, the points of contact of the tangents are:

$$\mathbf{q}_1 = \begin{pmatrix} \frac{16}{7} \\ 4\sqrt{33} \\ \frac{7}{7} \end{pmatrix}, \quad \mathbf{q}_2 = \begin{pmatrix} \frac{16}{7} \\ -4\sqrt{33} \\ \frac{7}{7} \end{pmatrix}$$

C Code

```
#include <stdio.h>
#include <math.h>

// Tangent function to be called from Python
void tangent_points(double r, double px, double py, double *Ax,
    double *Ay, double *Bx, double *By) {
    double ox = 0.0, oy = 0.0;
    double d = sqrt((px - ox)*(px - ox) + (py - oy)*(py - oy));

    if (d <= r) {
        *Ax = *Ay = *Bx = *By = NAN;
        return;
    }

    double r2 = r*r;
    double d2 = d*d;
    double k = r2 / d2;
    double h = (r / d2) * sqrt(d2 - r2);
```



```
1  *Ax = k*px - h*py;  
2  *Ay = k*py + h*px;  
3  *Bx = k*px + h*py;  
4  *By = k*py - h*px;  
5  }
```

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from numpy import linalg as LA
4
5 # Local imports
6 from libs.line.funcs import line_gen
7 from libs.conics.funcs import circ_gen
8
9 # --- Circle and external point ---
10 O = np.array([[0.0], [0.0]]) # Centre
11 r = 4.0 # Radius
12 P = np.array([[7.0], [0.0]]) # External point
13
14 # --- Tangent point calculation ---
15 u = P - O
16 d = float(LA.norm(u))
17 u_hat = u / d
```

Python Direct

```
# Rotation matrix for 90 degrees
R90 = np.array([[0, -1], [1, 0]])

# Tangent points formula
r2 = r**2
sqrt_term = np.sqrt(d**2 - r**2)

A = 0 + (r2/d)*u_hat + (r/d)*sqrt_term*(R90 @ u_hat)
B = 0 + (r2/d)*u_hat - (r/d)*sqrt_term*(R90 @ u_hat)

# --- Generate required curves ---
x_circ = circ_gen(0, r)
x_tan1 = line_gen(P, A)
x_tan2 = line_gen(P, B)
x_OP = line_gen(0, P)

# --- Plotting ---
plt.plot(x_circ[0, :], x_circ[1, :])
```

```
plt.plot(x_tan1[0, :], x_tan1[1, :], color='orange' )
plt.plot(x_tan2[0, :], x_tan2[1, :], color='green' )
plt.plot(x_OP[0, :], x_OP[1, :], 'k--')

# --- Mark and label points (rounded neatly) ---
points = {'O': O, 'P': P, 'A': A, 'B': B}
for name, pt in points.items():
    x, y = float(pt[0]), float(pt[1])
    plt.scatter(x, y, s=45, zorder=5)
    plt.text(x + 0.25, y + 0.25,
             f'{name}({round(x,2)}, {round(y,2)})',
             fontsize=9, ha='center')
```

```
# --- Axes setup ---
ax = plt.gca()
ax.spines['top'].set_color('none')
ax.spines['right'].set_color('none')
ax.spines['left'].set_position('zero')
ax.spines['bottom'].set_position('zero')

# --- Style ---
plt.axis('equal')
plt.grid(True)
plt.xlabel('$x$')
plt.ylabel('$y$')
plt.title('Tangents from an External Point to a Circle')
plt.legend(loc='upper right')
plt.show()
```

```
import ctypes
import numpy as np
import matplotlib.pyplot as plt
from libs.line.funcs import line_gen
from libs.conics.funcs import circ_gen

# Load the shared library
lib = ctypes.CDLL('./libtangent.so')

# Define argument and return types
lib.tangent_points.argtypes = [
    ctypes.c_double, ctypes.c_double, ctypes.c_double,
    ctypes.POINTER(ctypes.c_double), ctypes.POINTER(ctypes.
        c_double),
    ctypes.POINTER(ctypes.c_double), ctypes.POINTER(ctypes.
        c_double)
]
```

```
# Inputs
r = 4.0
P = np.array([[7.0], [0.0]])

# Prepare output containers
Ax = ctypes.c_double()
Ay = ctypes.c_double()
Bx = ctypes.c_double()
By = ctypes.c_double()

# Call the C function
lib.tangent_points(r, float(P[0]), float(P[1]),
                  ctypes.byref(Ax), ctypes.byref(Ay),
                  ctypes.byref(Bx), ctypes.byref(By))

# Convert results to numpy arrays
O = np.array([[0.0], [0.0]])
A = np.array([[Ax.value], [Ay.value]])
```

```
B = np.array([[Bx.value], [By.value]])

print(Tangent Points from C library:)
print(fA({A[0,0]:.2f}, {A[1,0]:.2f}))
print(fB({B[0,0]:.2f}, {B[1,0]:.2f}))
# Plot using your libs
x_circ = circ_gen(0, r)
x_tan1 = line_gen(P, A)
x_tan2 = line_gen(P, B)
x_OP = line_gen(0, P)
plt.plot(x_circ[0, :], x_circ[1, :], label='Circle')
plt.plot(x_tan1[0, :], x_tan1[1, :], color='orange', label='
    Tangent 1')
plt.plot(x_tan2[0, :], x_tan2[1, :], color='green', label='
    Tangent 2')
plt.plot(x_OP[0, :], x_OP[1, :], 'k--', label='$OP$')
# Mark and label points
points = {'O': 0, 'P': P, 'A': A, 'B': B}
for name, pt in points.items():
```



```
x, y = float(pt[0]), float(pt[1])
plt.scatter(x, y, s=45, zorder=5)
plt.text(x + 0.25, y + 0.25, f'{name}({round(x,2)}, {round(y
    ,2)})',
        fontsize=9, ha='center')
# -style axes
ax = plt.gca()
ax.spines['top'].set_color('none')
ax.spines['right'].set_color('none')
ax.spines['left'].set_position('zero')
ax.spines['bottom'].set_position('zero')
plt.axis('equal')
plt.grid(True)
plt.xlabel('$x$')
plt.ylabel('$y$')
plt.title('Tangents from External Point (using C library)')
plt.legend(loc='upper right')
plt.show()
```

Plot

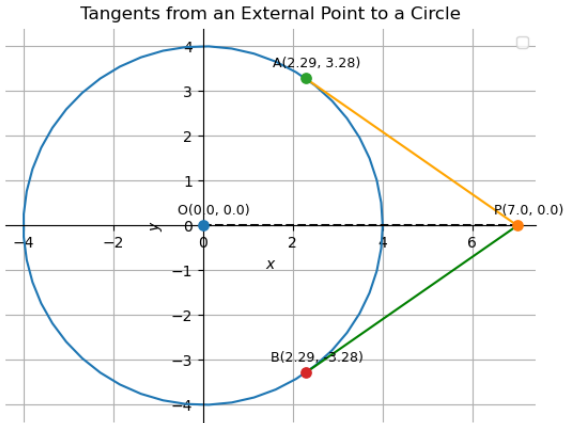


Figure: